

Skill or Skip? Learning Selective Skill Invocation in Agentic Tasks via Dual-Granularity Preference Learning

Chishui Chen^{1,2*} Jiaye Lin^{2,*†} Te Sun^{3*} Yi Yang^{2,4*}
Junxi Wang¹ Cong Qin^{2,5} Yangen Hu² Lu Pan² Ke Zeng²

¹Fudan University ²Meituan LongCat Interaction Team ³Shanghai Jiao Tong University

⁴Nanjing University ⁵Peking University

cschen26@m.fudan.edu.cn jiaye.lin22@gmail.com

Abstract

Agent skills are callable procedural modules that provide reusable knowledge and execution policies for complex agentic tasks. However, existing methods mainly focus on selecting relevant skills or improving the skills themselves, while overlooking whether a relevant skill should actually be invoked at the current decision point. Unhelpful invocations may introduce irrelevant context and disrupt an otherwise correct execution process. To address this issue, we propose **SelSkill**, a dual-granularity preference-learning framework for selective skill invocation. SelSkill formulates skill use as a *skill-or-skip* decision, uses predictive uncertainty to prioritize candidate decision points, and constructs controlled invoke-skip preference pairs from shared trajectory prefixes. It further combines episode-level outcome preferences with step-level invocation preferences to capture both overall trajectory quality and the local effectiveness of skill invocation. On ALF-World with Qwen3-8B, SelSkill improves task success by **10.9** points over the skill-enabled baseline and **7.8** over No-Skill, with **29.1**-point higher execution precision. On BFCL, task success and execution precision improve by **5.7** and **29.5** points, respectively. Zero-shot results on Tau-bench and PopQA suggest partial transfer to unseen domains and skills. Code: <https://github.com/ChenChiShui/selective-skill-invocation>.

1 Introduction

As agent systems are increasingly applied to long-horizon, highly interactive tasks, relying on the model to plan and execute from scratch at each step can underuse prior experience and lead to inefficient exploration (Wang et al., 2025b). In this context, agent skills have received growing attention in settings such as web interaction and software

engineering (Wang et al., 2025a; Ling et al., 2026; Li et al., 2026). As callable procedural modules, agent skills encapsulate domain knowledge, applicability conditions, and reusable execution policies, providing agents with reusable support for complex task solving (Jiang et al., 2026; Wang et al., 2026).

Existing methods mainly focus on either identifying useful skills or improving their construction and use. Some retrieve relevant skills from a library based on the current task context (Zheng et al., 2026; Su et al., 2026), while others construct or refine skills from external knowledge and interaction trajectories, sometimes integrating them into agent policy optimization (Xia et al., 2026; Tu et al., 2026). However, these methods largely assume that relevant skills should be invoked, while overlooking a more fundamental question: even if a skill is relevant, should the agent actually invoke it at the current decision point? During task execution, unhelpful invocations may introduce irrelevant context, thereby disrupting an otherwise correct execution process. Through our analysis, we reveal two important characteristics of skills during task execution:

(I) Highly Concentrated Skill Benefits. Figures 1(a)–(b) show that effective skill use does not mean invoking a skill whenever it appears relevant. The case study in Figure 1(a) illustrates that a seemingly relevant skill call can still produce an unnecessarily broad and suboptimal response. Across multiple benchmarks, the counterfactual results in Figure 1(b) show that enabling skill access improves the final outcome in only about **14%** of paired trajectories, has no clear effect in about **78%**, and worsens the outcome in about **8%**. Further analysis shows that harmful invocations are often semantically close to effective skill uses in the same context. Thus, *the value of a skill is concentrated in a narrow set of states, requiring precise invocation rather than calling skills whenever they appear relevant.*

*Equal contribution.

†Corresponding author.

Work done during an internship at Meituan.

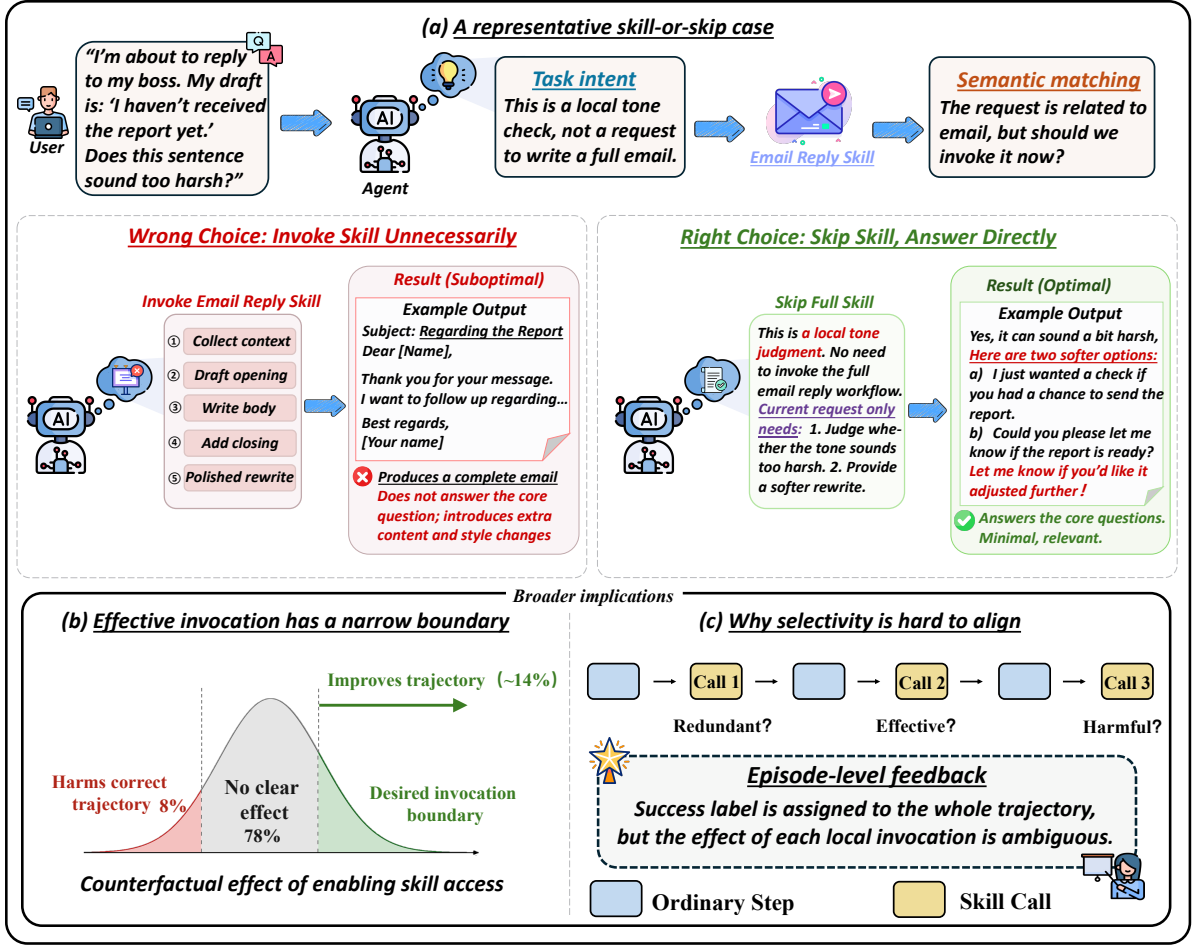


Figure 1: **Motivation for selective skill invocation.** (a) A representative skill-or-skip case illustrates that a relevant skill may still be unnecessary for the current request. (b) Counterfactual analysis shows that beneficial effects of skill access are concentrated in only a small fraction of paired trajectories. (c) Episode-level feedback cannot directly identify the local contribution of each invocation.

(II) Trajectory-Level Ambiguity. Figure 1(c) suggests that episode-level feedback alone makes it difficult to determine the effect of an individual skill invocation. For example, a skill call may help complete the task, act as an unhelpful step, or have its negative effect corrected by later actions. The final outcome does not reliably indicate whether invoking a skill was helpful at the current decision point. This makes it insufficient to learn skill invocation only from episode-level feedback or to address this problem with simple retrieval-based rules. Thus, *the effect of a skill invocation should be assessed at the decision-point level.*

Therefore, learning an effective skill invocation policy requires accounting for both the concentrated benefits of skill use and the ambiguity of trajectory-level feedback. This calls for learning signals that capture not only the overall utility of skill use for task completion, but also the local effectiveness of invoking a skill at a specific de-

cision point. To this end, we propose **SelSkill**, a preference-learning framework for selective skill invocation. SelSkill uses the model’s predictive uncertainty to guide the selection of candidate skill-decision points, and constructs contrastive training pairs by comparing skill invocation with skipping at these points. Furthermore, SelSkill combines episode-level outcome preferences with step-level invocation preferences, enabling the agent to more accurately determine when to invoke a skill and when to skip it. In summary, the main contributions of this paper are as follows:

- **Systematic Analysis.** We provide a detailed analysis of the limitations of existing skill invocation methods and formulate selective skill invocation as a *skill-or-skip* problem at each decision point, determining whether the agent should invoke a skill under the current state.
- **Novel Optimization Framework.** We propose

SelSkill, a preference-learning framework for selective skill invocation, which optimizes the skill invocation policy by constructing invoke-skip contrastive pairs and combining episode-level and step-level preferences.

- **Strong Empirical Results.** On ALFWorld, SelSkill improves task success by **10.9** points over the skill-enabled baseline and **7.8** points over No-Skill, while improving execution precision by **29.1** points. On BFCL, it improves task success and execution precision by **5.7** and **29.5** points, respectively.

2 Related Work

2.1 From Tools and Experience to Skills

Language agents often use external tools and past experience to extend the base model. Prior work studies API invocation, function selection, and argument generation (Schick et al., 2023; Patil et al., 2024). Later work abstracts tool chains or interaction traces into reusable procedural representations (Chen et al., 2026). Building on these abstractions, skills provide a compact form of reusable experience while retaining part of the executability of tools. They package domain knowledge, applicability conditions, and executable or textual procedures (Jiang et al., 2026; Ling et al., 2026). They can also be organized into structured libraries to support retrieval and controlled injection during agent execution (Wang et al., 2026).

2.2 Skill Integration and Optimization

Existing skill-based agent methods mainly utilize skills in three ways. First, routing-based methods address skill selection in large libraries by matching the current context to a small set of candidate skills using routers, retrievers, or graph-based representations (Zheng et al., 2026; Liang et al., 2026; Liu et al., 2026). Second, skill-library management methods maintain and expand the library by adding, revising, or pruning skills based on environment interaction (Yang et al., 2026; Ni et al., 2026; Mi et al., 2026; Ouyang et al., 2026). Third, building on dynamically maintained skill or experience libraries, reinforcement-learning methods use retrieved reusable knowledge to guide exploration and provide behavior priors during policy optimization (Xia et al., 2026; Tu et al., 2026; Lu et al., 2026; Shi et al., 2026). However, existing work mainly studies how to obtain, maintain, or use skills, while a relevant skill may still be un-

necessary or harmful at a specific decision point, a concern also noted in recent analyses of skill-based agents (Li et al., 2026; Su et al., 2026).

2.3 Selectivity in Related Agent Settings

Related studies have identified selectivity as a concern in agents that access external resources. In tool-augmented agents, models may invoke tools when they are not helpful or fail to use tool results effectively (Chen et al., 2025; Xu et al., 2025; Ross et al., 2025). In memory-augmented agents, retrieved experience may not match the current task context (Xiong et al., 2025). Concurrent work further learns proactive retrieval over an evolving experience base through paired retrieval/no-retrieval rollouts (Cai et al., 2026). These studies collectively suggest that external assistance should not be used indiscriminately in practice.

3 Preliminary

We consider an agent that performs tasks in a multi-step environment. At step t , the agent conditions on a trajectory prefix h_t and generates an action a_t . The prefix h_t may include the task instruction, interaction history, environment observations, and previously returned tool or skill outputs.

In addition to ordinary actions, the agent has access to a fixed skill library $\mathcal{S}$. Each skill $s \in \mathcal{S}$ is a callable procedural module with lightweight metadata m_s and full skill content c_s . The metadata includes the skill name and a short description, while the full content contains reusable knowledge, constraints, procedures, or action policies. We denote the visible metadata of the skill library as $M_S = \{m_s : s \in \mathcal{S}\}$. At decision time, the agent policy is written as $\pi_\theta(a_t | h_t, M_S)$, where a_t can be either an ordinary environment action or a skill invocation. The full skill content c_s is not injected into the model context by default; it is loaded or executed only after the model explicitly invokes the corresponding skill. Specifically, *memory skills* return textual information such as strategy hints or API documentation, while *executable skills* encapsulate action or tool-call sequences. This paper does not study how to generate, modify, or improve the skills themselves. Instead, given a fixed skill library, we study *selective skill invocation*: deciding whether and when a relevant skill should intervene in a multi-step trajectory.

For each benchmark, the skill library is constructed offline from the training split and remains

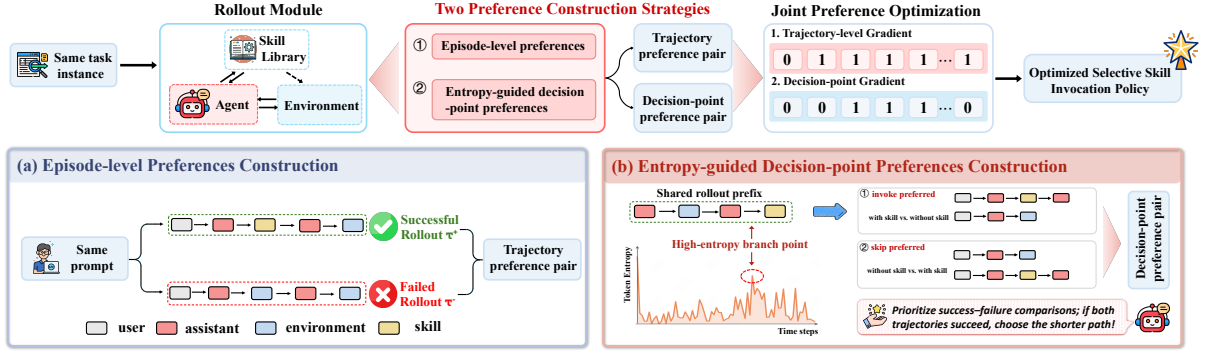


Figure 2: **The overview of SelSkill.** We construct episode-level trajectory preferences and entropy-guided decision-point preferences, and jointly optimize the policy for selective skill invocation.

fixed throughout training and evaluation; construction details are provided in Appendix A.

4 Methodology

4.1 Overview

The overview of SelSkill is illustrated in Figure 2. Our framework constructs two complementary preference signals, namely episode-level preferences and local decision-point preferences. These two signals guide the agent to balance overall task utility with the local effectiveness of skill invocation, as detailed in the following subsections.

4.2 Preference Construction

Episode-level preferences. Episode-level preferences provide a global task-outcome signal. For the same task, we sample multiple complete trajectories and group them according to final task success. If one trajectory succeeds and another fails, we construct a preference pair:

$$(\tau^+, \tau^-), \quad (1)$$

where τ^+ denotes a successful trajectory and τ^- denotes a failed trajectory. This pair indicates that the model should prefer the complete behavior sequence that solves the task.

This signal constrains the overall downstream utility of skill invocation. It does not directly determine whether an individual skill call is necessary, but it identifies which complete trajectories are ultimately more effective.

Local decision-point preferences. A limitation of episode-level preferences is that they only provide trajectory-level success or failure feedback, making it difficult to assign credit to a specific skill invocation decision. To directly optimize local invocation decisions, we further construct local

decision-point preferences. Specifically, for each rollout, we record token-level log-probabilities during generation and compute the predictive entropy at candidate skill-decision points. These candidate points cover uncertain states both after skill invocation and during ordinary generation. Given a trajectory prefix h_t , the predictive entropy is defined as:

$$H(h_t) = - \sum_v p_\theta(v | h_t) \log p_\theta(v | h_t), \quad (2)$$

where v denotes a token in the vocabulary. A higher entropy indicates greater uncertainty in the model’s subsequent generation. Motivated by prior findings that tool interactions can produce high-entropy decision points suitable for targeted branching (Dong et al., 2025; Chen et al., 2025), we use entropy to prioritize candidate positions for local invoke/skip comparison during pair construction.

We further examine this heuristic through an entropy-fork analysis on ALFWorld (Shridhar et al., 2021). As a diagnostic analysis, we create invoke/skip forks at actual skill-call positions and compare the token-level entropy of the two paths after the fork, as shown in Figure 3. The invoke path with skill injection exhibits higher average token entropy in subsequent action prediction. This suggests that skill injection often increases the model’s uncertainty when integrating the returned skill information into the next actions. We therefore use predictive entropy as a lightweight signal to prioritize candidate branch points that are more likely to produce informative invoke/skip comparisons.

For each selected skill-decision point (h_t, s) , we construct two continuations: one that invokes skill s and one that skips it. Both continuations start from the same prefix h_t and differ only in the forced local decision: invoking skill s or skipping it. We roll out both branches to the end of the episode

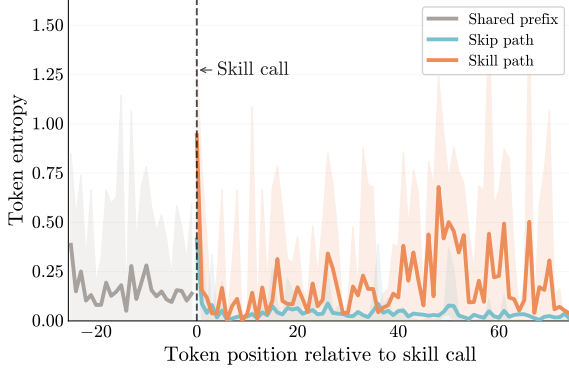


Figure 3: **Token entropy around invoke/skip points.**

and assign labels by an outcome-efficiency rule. A successful continuation is preferred over a failed one. If both continuations succeed, we prefer the shorter one, measured by the number of environment steps after the branch. If both continuations fail, we discard the branch. This keeps redundant-but-successful skill calls in the training signal and encourages the model to skip unnecessary skills.

$$(c_t^+, c_t^- \mid h_t, s), \quad (3)$$

where c_t^+ and c_t^- denote the preferred and dis-preferred continuations, respectively, under this outcome-efficiency ordering.

This construction differs from episode-level pairing because the two continuations share the same history h_t and are generated by an explicit invoke/skip intervention at the branch point. Thus, although the label is still evaluated by downstream outcome and efficiency, the comparison controls for the pre-branch trajectory and isolates the immediate skill-or-skip choice more directly than pairing independently sampled complete trajectories.

To focus this signal on the local invoke/skip decision, we compute the DPO loss only within a local window after the branching point. Specifically, we apply a local loss mask $M_t^{(n)}$ to the continuation and keep only the next n assistant turns after the branch for local DPO. This makes the gradients more directly target the short-term consequences of the invoke/skip decision rather than the entire continuation of each branch.

Local decision-point preferences concentrate the learning signal around specific invoke/skip decisions, while episode-level preferences provide full-trajectory constraints.

4.3 Preference Optimization

We optimize the constructed preference data using Direct Preference Optimization (DPO) (Rafailov

et al., 2023). For a conditioning input z and an output y , we define $r_\theta(z, y) = \log \pi_\theta(y \mid z) - \log \pi_{\text{ref}}(y \mid z)$, where π_θ is the trainable model and π_{ref} is the reference model. Given a preference pair (y^+, y^-) , the DPO loss is

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(z, y^+, y^-) \sim \mathcal{D}} \left[\log \sigma \left(\beta r_\theta(z, y^+) - \beta r_\theta(z, y^-) \right) \right], \quad (4)$$

where β is the DPO temperature.

We merge episode-level preference pairs and local decision-point preference pairs into the training data and optimize them with the same DPO objective. For episode-level pairs, z is the task context, while y^+ and y^- are the successful and failed trajectories. For local pairs, the model is conditioned on the pre-branch history h_t and the full skill meta-data list, while s only identifies the candidate skill used to create the forced invoke/skip fork. The paired continuations (y^+, y^-) are then ordered by the outcome-efficiency rule. During optimization, $M_t^{(n)}$ masks out tokens outside the first n assistant turns after the branch, so local DPO trains only on the selected assistant-generated tokens.

5 Experiments

5.1 Experimental Setup

Benchmarks. We evaluate SelSkill on four benchmarks. ALFWorld (Shridhar et al., 2021) evaluates multi-step embodied decision making, and BFCL (Patil et al., 2025) evaluates multi-turn function calling. We further use PopQA (Mallen et al., 2023) and Tau-bench (Yao et al., 2024) to evaluate out-of-domain transfer.

Backbones. We use models with basic task-solving ability and valid skill-call formats, allowing SelSkill to focus on selective invocation. For ALFWorld, since raw models are unstable in environment interaction, we train no-skill Qwen3-4B/8B (Yang et al., 2025) policies with GRPO (Shao et al., 2024) as **RL-Init**, and enable skills afterward. For BFCL, we use Qwen3-14B as **Base**, which already supports reliable function-style skill calls through prompting without additional RL initialization. We follow standard benchmark-specific evaluation protocols and configurations. Details are provided in Appendix G.

Baselines. We compare SelSkill with No-Skill, skill-enabled baselines without selective-

invocation training (**RL-Init w/ Skill** for ALFWorld and **Base w/ Skill** for BFCL), and signal ablations (Episode only, Entropy-local only, and Skill-call-local only). For out-of-domain evaluation, we compare No-Skill with +Skill, which enables target-benchmark skills without additional training. Appendix D reports additional engineering and adapted selective-invocation baselines.

Metrics. We report the following metrics. **SR** measures the task success rate. **Exec. Prec.** measures whether a skill invocation is valid and successfully completed. It is the fraction of attempted invocations that are valid and successfully completed. Invalid, malformed, and execution-error calls count as failures. For executable skills, this means the skill can interact with the environment without execution errors caused by unmet preconditions, invalid arguments, or malformed calls; for memory skills, this means the call returns valid content for subsequent generation. **SR@Invoke** and **SR@Skip** report success rates conditioned on whether an episode invokes at least one skill. Their changes provide an indirect view of how selectively the model invokes skills across different trajectory states. We additionally report **Skill/ep** as an indicator of invocation frequency and **Avg. Steps** as an indicator of trajectory efficiency.

5.2 Main Results

Performance on ALFWorld. Table 1 reports the main results on ALFWorld. The experimental results can be summarized in three points:

(i) **Necessity of selective invocation.** Simply enabling skills on RL-Init does not lead to consistent performance gains: it improves task success in some settings but degrades it in others. This inconsistency suggests that the model cannot yet reliably determine when and how to invoke skills.

(ii) **Improved invocation reliability.** Compared with skill-enabled initialization, SelSkill substantially improves execution precision and shortens trajectories overall. For Qwen3-8B, Exec. Prec. increases from 70.9% to 100.0%, while Avg. Steps decreases from 24.0 to 16.9 by Round3.

(iii) **A narrow but reliable invocation boundary.** For Qwen3-8B, SelSkill Round3 achieves 97.0% SR@Invoke while reducing Skill/ep from 2.55 to 0.44, suggesting that the model learns a more selective and reliable invocation policy. Qwen3-4B likewise improves SR and SR@Invoke, while keeping its invocation frequency close to the

skill-enabled initialization.

Performance on BFCL. Table 1 also reports the main results on BFCL. The experimental results can be summarized in three points:

(i) **Incremental value of skill access.** Enabling skills improves SR from 14.1% to 18.5%, showing that skills can provide additional value in multi-turn function-calling tasks, while leaving room for learning more reliable invocation behavior.

(ii) **Execution precision as a key bottleneck.** **Base w/ Skill** achieves only 44.0% Exec. Prec., indicating that many skill calls are not validly and successfully executed.

(iii) **Higher-quality invocation.** Compared with **Base w/ Skill**, SelSkill Round2 improves SR from 18.5% to 24.2% and Exec. Prec. from 44.0% to 73.5%, while reducing Avg. Steps from 18.2 to 14.2. Notably, Skill/ep increases from 0.73 to 1.01 rather than decreasing. This suggests that SelSkill does not simply suppress skill use; instead, it enables more reliable invocations, higher task success, and shorter trajectories. Appendix D provides a concise BFCL failure breakdown.

5.3 Ablation Study

We conduct ablation experiments on ALFWorld with Qwen3-8B to examine the contributions of different preference signals. All variants use the same one-round training setting as **SelSkill Round1**. SelSkill combines episode-level preferences, which compare successful and failed trajectories, with entropy-guided step-level preferences, which supervise local invoke/skip decisions. The loss-mask parameter n controls how many post-branch assistant turns are included in the local training objective. Table 2 reports the results, which can be summarized in three points:

(i) **Episode-level supervision is insufficient.** **Episode only**, which corresponds to standard DPO using only episode-level preference pairs, achieves 75.0% SR and 75.4% Exec. Prec. This suggests that trajectory-level preference learning alone cannot reliably supervise local skill invocation decisions.

(ii) **Local supervision alone is unbalanced.** **Entropy-local only** achieves relatively high Exec. Prec. but low SR, while **Skill-call-local only** improves SR to 80.5% but reduces Exec. Prec. to 41.7% with 4.29 Skill/ep. This indicates that local supervision alone may improve one aspect of invocation behavior while sacrificing others.

(iii) **Mixed signals achieve the best balance.**

Method	SR (↑)	Exec. Prec. (↑)	SR@Invoke (↑)	SR@Skip (↑)	Skill/ep	Avg. Steps (↓)
<i>ALFWorld benchmark</i>						
Qwen3-4B (No-Skill)	66.4	—	—	—	—	26.2
+RL-Init w/ Skill	69.5	87.8	81.4	63.5	0.58	23.8
+SelSkill Round1	73.4	81.6	86.8	64.0	0.68	21.5
+SelSkill Round2	77.3	96.0	92.6	66.2	0.59	21.3
Qwen3-8B (No-Skill)	78.9	—	—	—	—	22.3
+RL-Init w/ Skill	75.8	70.9	73.7	78.8	2.55	24.0
+SelSkill Round1	82.8	94.1	90.7	78.8	0.66	19.7
+SelSkill Round2	85.9	96.6	91.4	83.9	0.46	16.3
+SelSkill Round3	86.7	100.0	97.0	83.2	0.44	16.9
<i>BFCL benchmark</i>						
Qwen3-14B (No-Skill)	14.1	—	—	—	—	24.2
+Base w/ Skill	18.5	44.0	13.4	22.8	0.73	18.2
+SelSkill Round1	22.6	69.7	14.8	30.8	0.92	14.1
+SelSkill Round2	24.2	73.5	18.2	32.4	1.01	14.2

Table 1: **Performance comparison of different baselines on ALFWorld and BFCL benchmarks.** Blue denotes our methods. For each backbone, the best results are highlighted in **bold**.

Setting	SR (↑)	Skill/ep	Exec. Prec. (↑)
<i>Episode-level</i>			
Episode only (standard DPO)	75.0	1.30	75.4
<i>Step-level</i>			
Entropy-local only	70.3	1.77	81.0
Skill-call-local only	80.5	4.29	41.7
<i>Episode-level + Step-level</i>			
SelSkill ($n = 1$)	79.7	0.80	91.2
SelSkill ($n = 3$)	82.8	0.66	94.1
SelSkill ($n = \text{all}$)	82.0	0.72	92.6

Table 2: **Ablation study on ALFWorld benchmark using Qwen3-8B.** Red denotes mixed-signal training, and n denotes the number of post-branch assistant turns covered by the loss mask.

SelSkill with $n = 3$ achieves the highest SR of 82.8% and Exec. Prec. of 94.1%, with 0.66 Skill/ep. These results show that combining episode-level and step-level preferences improves both task success and invocation quality, rather than relying on more frequent skill calls.

5.4 Out-of-Domain Generalization

A key question is whether the selective invocation ability learned on BFCL can generalize to new domains with previously unseen skills. We evaluate BFCL SelSkill Round2 in a zero-shot manner on two OOD benchmarks that are not seen during training. PopQA uses Wikipedia retrieval skills for knowledge-intensive question answering, while Tau-bench evaluates multi-turn service-oriented agent tasks with benchmark-specific skills and a GPT-4.1 user simulator. Tables 3 and 4 report the

results on PopQA and Tau-bench, respectively. The results can be summarized in three points:

(i) **Partial transfer across domains and unseen skills.** Neither benchmark uses the BFCL skills available during training, so this evaluation tests the transfer of *invocation judgment* rather than memorization of specific skill knowledge. On both OOD benchmarks, enabling benchmark-specific skills improves performance over the no-skill baseline, showing that the newly provided skills are useful in their target domains. SelSkill further improves over +Skill on both benchmarks, suggesting partial transfer of its invocation criterion.

(ii) **Selective retrieval on PopQA.** In this setting, Skill Rate denotes the percentage of examples where the model invokes a Wikipedia retrieval skill. Since PopQA is single-turn, Skill Rate is equivalent to **Skill/ep**. We use entity popularity as a rough proxy for how likely the answer is to be covered by the model’s parametric knowledge: high-popularity entities are more likely to be internalized by the model, while mid- and low-popularity entities typically require external retrieval more often. Table 3 shows that SelSkill improves EM across all popularity groups while reducing Skill Rate in an intuitive way. For high-popularity entities, Skill Rate drops substantially from 78% to 57%, while EM improves from 58.4 to 60.2. In contrast, for mid- and low-popularity entities, SelSkill only slightly reduces Skill Rate, from 93% and 94% to 87%, while improving EM from 62.0/62.7 to 63.9/64.5. This indicates that SelSkill does not simply suppress skill use; instead, it tends to preserve retrieval

Method	Overall EM ($\uparrow$)	High-pop		Mid-pop		Low-pop	
		EM ($\uparrow$)	Skill Rate	EM ($\uparrow$)	Skill Rate	EM ($\uparrow$)	Skill Rate
Base	20.1	42.2	—	8.4	—	9.6	—
+Skill	61.0	58.4	78%	62.0	93%	62.7	94%
SelSkill	62.9	60.2	57%	63.9	87%	64.5	87%

Table 3: OOD transfer on PopQA benchmark.

Method	Avg. ($\uparrow$)	Air. ($\uparrow$)	Ret. ($\uparrow$)	SR@Inv./Skip ($\uparrow$)
Base	31.9	22.0	41.7	— / —
+Skill	39.6	34.0	45.2	41.7 / 47.8
SelSkill	41.4	34.0	48.7	50.0 / 49.2

Table 4: OOD transfer on Tau-bench. Avg. is mean pass@1 over airline (Air.) and retail (Ret.) domains.

for examples that likely require external knowledge and skip unhelpful calls when parametric knowledge is more likely to suffice.

(iii) More reliable invocation on Tau-bench.

On Tau-bench, we evaluate SelSkill zero-shot with unseen APIs, policies, and skills. Compared with +Skill, it improves average pass@1 from 39.6 to 41.4. Both Avg. and SR@Invoke are macro-averaged across airline and retail; the latter increases from 41.7 to 50.0, indicating that invoked episodes succeed more often in new settings.

6 Analyses

Gradient localization. To examine whether step-level preferences provide more localized supervision for skill invocation, we compare gradient peak positions between ALFWorld episode-level pairs from complete trajectories and entropy-guided step-level pairs branched at high-uncertainty skill-decision points. For each pair, we align the skill-call position to 0, record the token position with the largest gradient norm, and visualize its distribution using kernel density estimation. The vertical axis in Figure 4 represents the estimated density of gradient peak positions.

Figure 4 shows that episode-level preferences produce more dispersed gradient peaks, whereas step-level preferences concentrate them around the skill-call region. This suggests that episode-level preferences provide broad trajectory-level guidance, while step-level preferences more directly supervise local invoke/skip decisions. The two signals thus provide complementary supervision for selective skill invocation, consistent with the advantage of mixed-signal training in Table 2.

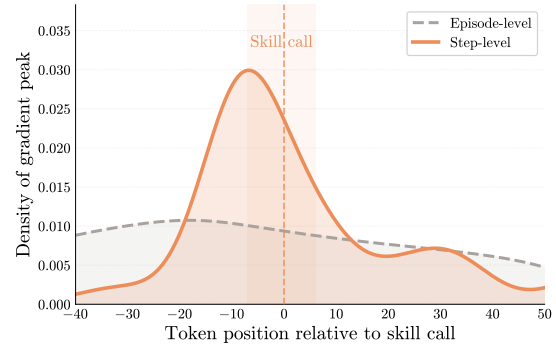


Figure 4: Step-level preferences produce more localized gradient peaks around the skill-call token.

Additional analyses. Appendix B uses counterfactual comparisons to examine when skill calls are useful. Skill invocation improves only a small fraction of trajectories, and harmful calls can still be semantically plausible. Appendix C examines episode-level reinforcement learning and finds that it does not reliably calibrate local invocation decisions. Skill usage remains unstable even when overall task success improves. Appendix D compares SelSkill with engineering and adapted selective-invocation baselines. These simple alternatives do not replace selectivity training. Appendix E evaluates entropy-guided branching. It selects more informative local comparisons than random branching and achieves comparable performance to all-skill branching at lower sampling cost. Appendix F tests robustness under injected distractor skills. SelSkill almost never invokes noise skills and shows only modest degradation with the largest skill listing. Finally, Appendix H presents cases in which skills are unnecessary, invoked before their preconditions are met, or beneficial when invoked at the appropriate time.

7 Conclusion

We formulate selective skill invocation as a *skill-or-skip* decision and propose **SelSkill**, which learns selective invocation policies from combined episode- and step-level preferences. On ALFWorld, SelSkill improves task success by **10.9** points over the skill-enabled baseline and **7.8** points over No-

Skill, while improving execution precision by **29.1** points. On BFCL, task success and execution precision improve by **5.7** and **29.5** points, respectively. Zero-shot results on PopQA and Tau-bench suggest partial transfer to unseen domains and skills.

Limitations

This work studies selective skill invocation with a fixed, offline-constructed skill library, while its construction and evolution are beyond our scope. We use predictive entropy as a lightweight heuristic for prioritizing candidate decision points, leaving alternative criteria to future work. Our evaluation is limited to Qwen3 backbones. Finally, our benchmarks do not fully capture deployments with irreversible external effects.

References

- Yuxuan Cai, Jie Zhou, Qin Chen, and Liang He. 2026. [Ask only when needed: Proactive retrieval from memory and skills for experience-driven lifelong agents](#). *CoRR*, abs/2604.20572.
- Shiqi Chen, Jingze Gai, Ruochen Zhou, Jinghan Zhang, Tongyao Zhu, Junlong Li, Kangrui Wang, Zihan Wang, Zhengyu Chen, Klara Kaleb, Ning Miao, Siyang Gao, Cong Lu, Manling Li, Junxian He, and Yee Whye Teh. 2026. [Skillcraft: Can LLM agents learn to use tools skillfully?](#) *CoRR*, abs/2603.00718.
- Yifei Chen, Guanting Dong, and Zhicheng Dou. 2025. [Toward effective tool-integrated reasoning via self-evolved preference learning](#). *CoRR*, abs/2509.23285.
- Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. 2025. [Agentic reinforced policy optimization](#). *CoRR*, abs/2507.19849.
- Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. 2026. [Sok: Agentic skills - beyond tool use in LLM agents](#). *CoRR*, abs/2602.20867.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, and 22 others. 2026. [Skillsbench: Benchmarking how well agent skills work across diverse tasks](#). *Preprint*, arXiv:2602.12670.
- Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, and 30 others. 2026. [Skillnet: Create, evaluate, and connect AI skills](#). *CoRR*, abs/2603.04448.
- George Ling, Shanshan Zhong, and Richard Huang. 2026. [Agent skills: A data-driven analysis of claude skills for extending large language model functionality](#). *CoRR*, abs/2602.08004.
- Dawei Liu, Zongxia Li, Hongyang Du, Xiyang Wu, Shihang Gui, Yongbei Kuang, and Lichao Sun. 2026. [Graph of skills: Dependency-aware structural retrieval for massive agent skills](#). *CoRR*, abs/2604.05333.
- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. 2026. [SKILL0: in-context agentic reinforcement learning for skill internalization](#). *CoRR*, abs/2604.02268.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. [When not to trust language models: Investigating effectiveness of parametric and non-parametric memories](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2023, Toronto, Canada, July 9-14, 2023, pages 9802–9822. Association for Computational Linguistics.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. 2026. [Skill-pro: Learning reusable skills from experience via non-parametric ppo for llm agents](#). *Preprint*, arXiv:2602.01869.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. 2026. [Trace2skill: Distill trajectory-local lessons into transferable agent skills](#). *CoRR*, abs/2603.25158.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. 2026. [Skillos: Learning skill curation for self-evolving agents](#). *Preprint*, arXiv:2605.06614.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. [The berkeley function calling leaderboard \(BFCL\): from tool use to agentic evaluation of large language models](#). In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. [Gorilla: Large language model connected with massive apis](#). In *Advances in Neural Information Processing Systems 38: Annual*

- Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024.*
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. [Direct preference optimization: Your language model is secretly a reward model](#). In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. 2025. [When2call: When \(not\) to call tools](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, pages 3391–3409. Association for Computational Linguistics.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [Deepseekmath: Pushing the limits of mathematical reasoning in open language models](#). *Preprint*, arXiv:2402.03300.
- Yaorui Shi, Yuxin Chen, Zhengxi Lu, Yuchun Miao, Shugui Liu, Qi GU, Xunliang Cai, Xiang Wang, and An Zhang. 2026. [Skill1: Unified evolution of skill-augmented agents via reinforcement learning](#). *Preprint*, arXiv:2605.06130.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew J. Hausknecht. 2021. [Alfworld: Aligning text and embodied environments for interactive learning](#). In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.
- Weihang Su, Jianming Long, Qingyao Ai, Yichen Tang, Changyue Wang, Yiteng Tu, and Yiqun Liu. 2026. [Skill retrieval augmentation for agentic AI](#). *CoRR*, abs/2604.24594.
- Songjun Tu, Chengdong Xu, Qichao Zhang, Yaocheng Zhang, Xiangyuan Lan, Linjing Li, Dong Li, and Dongbin Zhao. 2026. [Dynamic dual-granularity skill bank for agentic rl](#). *Preprint*, arXiv:2603.28716.
- Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. 2026. [Skillx: Automatically constructing skill knowledge bases for agents](#). *CoRR*, abs/2604.04804.
- Liang Wang, Nan Yang, Xiaolong Huang, Bin-xing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. [Text embeddings by weakly-supervised contrastive pre-training](#). *CoRR*, abs/2212.03533.
- Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. 2025a. [Inducing programmatic skills for agentic tasks](#). *CoRR*, abs/2504.06821.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025b. [Agent workflow memory](#). In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. [Skillrl: Evolving agents via recursive skill-augmented reinforcement learning](#). *Preprint*, arXiv:2602.08234.
- Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. 2025. [How memory management impacts llm agents: An empirical study of experience-following behavior](#). *Preprint*, arXiv:2505.16067.
- Hongshen Xu, Zihan Wang, Zichen Zhu, Lei Pan, Xingyu Chen, Shuai Fan, Lu Chen, and Kai Yu. 2025. [Alignment for efficient tool calling of large language models](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 17776–17792. Association for Computational Linguistics.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. [Qwen3 technical report](#). *Preprint*, arXiv:2505.09388.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. 2026. [Autoskill: Experience-driven lifelong learning via skill self-evolution](#). *CoRR*, abs/2603.01145.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. [\$\tau\$ -bench: A benchmark for tool-agent-user interaction in real-world domains](#). *CoRR*, abs/2406.12045.
- Yanzhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. 2026. [Skillrouter: Skill routing for LLM agents at scale](#). *CoRR*, abs/2603.22455.

Appendix

A Skill Library	11
B Counterfactual Skill Benefit	11
C Episode-Level RL Does Not Reliably Calibrate Skill Use	12
D Additional Selective-Invocation Baselines	13
E Ablation on Entropy-Guided Branching	13
F Robustness to Distractor Skills	14
G Experimental Details	15
G.1 Benchmark-Specific Setup	15
G.2 Preference Pair Collection	15
G.3 Training Rounds and Loss Masking	16
G.4 Compute Cost	16
H Trajectory Analysis	16

A Skill Library

We study selective skill invocation under a fixed skill library. All skills are constructed offline before policy optimization, using only training resources such as training trajectories, task instructions, API schemas, and environment documentation. These skills remain unchanged throughout SelSkill training and evaluation, and no evaluation examples are used during skill construction.

For each benchmark, we identify recurring procedural patterns from training resources and consolidate them into reusable callable skills. Each skill consists of lightweight metadata and full skill content. The metadata is visible to the model before invocation and supports the skill-or-skip decision, while the full content is injected into the context or executed only after the model explicitly chooses to invoke the skill.

Skill construction follows two principles. First, each skill should capture a reusable procedure that can apply across multiple task instances, such as checking preconditions, following environment constraints, retrieving supporting evidence, or executing a common action sequence. It should not encode an instance-specific solution or a shortcut tailored to a particular example. Second, skill content should avoid any evaluation leakage. It must

not contain evaluation answers, evaluation trajectories, target states, case identifiers, or other information that would allow the model to solve an evaluation instance by memorization rather than by deciding when to invoke a fixed skill.

Table 5 shows representative skills from each benchmark. The entries in the table are summarized and shortened; full skill definitions are provided in the supplementary materials.

B Counterfactual Skill Benefit

We conduct counterfactual experiments to characterize when skill access actually changes the execution trajectory. For each benchmark and model, we deliberately construct a counterfactual pair for the same task instance: a skill-disabled trajectory and a skill-enabled trajectory under the same task setting. We then align the two runs by case ID and compare whether enabling skills changes the final outcome. This design compares outcome changes under skill access on the same case, rather than comparing different tasks or different sampled instances.

The analysis covers 513 paired runs across multiple benchmarks and model families: BFCL and Tau-bench with Qwen3-14B, ALFWorld with Qwen3-8B, and BFCL with Gemini-2.5-flash-lite. For BFCL, we use the base and long-context multi-turn splits and exclude missing-function and missing-parameter categories, where the required function or parameter information is unavailable and skill use is therefore structurally blocked. For Tau-bench and ALFWorld, we align paired runs by task ID/trial and game file, respectively.

Table 6 shows that skill benefits are highly concentrated. Across 513 paired runs, enabling skills improves the trajectory in only 13.5% of cases, harms an otherwise correct trajectory in 8.4%, and has no clear effect in 78.2%. Thus, skill access is not uniformly beneficial: most cases either do not require the skill or cannot be changed by it, while a smaller but non-negligible set of cases is sensitive to the invocation decision.

Panel B further examines whether harmful invocations are simply caused by choosing semantically unrelated skills. We compute BM25 and embedding-based cosine similarity between the task instruction and the invoked skill metadata, using E5 (Wang et al., 2022) as the embedding model. Negative cases have similarity scores close to Positive cases and higher than No-clear-effect, indicating that harmful invocations are often seman-

Benchmark	Skills	Example	Before invocation	After invocation
ALFWorld	10 (4 / 6)	heat object	Use when a task requires a heated object.	Check object and microwave availability, then perform the heating procedure.
BFCL	18 (10 / 8)	place stock order	Use for stock order placement or verification.	Check ticker, side, quantity, order type, and price, while preserving user-specified limit prices.
Tau-bench airline	11 (4 / 7)	cancel flight	Use when a user requests flight cancellation.	Check cancellation eligibility under airline policy before calling the cancellation API.
Tau-bench retail	6 (2 / 4)	cancel order	Use when a user requests order cancellation.	Check order status, cancellation window, and item eligibility before calling the cancellation API.
PopQA	4 (4 / 0)	lookup person fact	Use when uncertain about a person’s biographical attribute.	Retrieve Wikipedia evidence and extract the requested attribute.

Table 5: **Representative skills from each benchmark.** Skill counts are reported as total (memory / executable). Entries are summarized and shortened.

Panel A: Counterfactual outcome changes					
Outcome	BFCL Qwen3 ($n = 124$)	ALFWorld Qwen3 ($n = 84$)	Tau-bench Qwen3 ($n = 181$)	BFCL Gemini ($n = 124$)	Total ($n = 513$)
Positive	21 (16.9%)	9 (10.7%)	34 (18.8%)	5 (4.0%)	69 (13.5%)
Negative	6 (4.8%)	1 (1.2%)	20 (11.0%)	16 (12.9%)	43 (8.4%)
No clear effect	97 (78.2%)	74 (88.1%)	127 (70.2%)	103 (83.1%)	401 (78.2%)

Panel B: Semantic relevance of invoked skills			
Outcome	BM25	Embedding cosine (E5)	Interpretation
Positive	4.58	0.721	Relatively relevant
Negative	4.19	0.719	Relatively relevant
No clear effect	3.62	0.703	Relatively less relevant

Table 6: **Counterfactual analysis of skill benefit.** Panel A reports outcome changes after enabling skills. Panel B reports similarity between task instructions and invoked skill metadata, showing that both helpful and harmful invocations can appear semantically relevant.

tically plausible. The BM25 gap between effective or harmful invocations and unnecessary invocations is significant ($p = 0.042$), while embedding similarity shows a consistent but non-significant trend. This suggests that many failures arise after the model has already selected a plausible skill: the harder problem is deciding whether the current state provides the right conditions and timing for invoking it effectively.

C Episode-Level RL Does Not Reliably Calibrate Skill Use

We compare two GRPO+KL variants on ALFWorld to examine whether episode-level reward can learn selective skill invocation. Both variants start from Qwen3-8B base and use the same training hyperparameters: learning rate $1e-6$, group

size 8, train batch size 32, mini-batch size 64, KL coefficient 0.01, maximum 50 steps per episode, and validation temperature 0.4. GRPO-w/OSKILL does not receive any skill listing and is trained with task reward only: +10 for success and -0.1 for invalid actions. GRPO-w/SKILL receives the skill listing at every step and uses the same task reward plus a skill-success bonus of +1.0 whenever a skill executes without error.

Table 7 shows that episode-level RL improves task success at some checkpoints, but does not produce stable skill-use behavior. The advantage of GRPO-w/SKILL over GRPO-w/OSKILL fluctuates across training, and skill calls per episode do not converge to a consistent pattern. Episodes without skill calls often achieve higher success rates than episodes with skill calls, suggesting that the

Step	w/o Skill SR ($\uparrow$)	w/ Skill SR ($\uparrow$)	Δ	Skill/ep	SR@Invoke ($\uparrow$)	SR@Skip ($\uparrow$)
0	19.5	21.4	+1.9	11.13	20.7	50.0
5	24.2	19.0	-5.2	1.25	24.3	14.3
10	26.6	39.1	+12.5	3.26	30.8	63.6
15	28.9	45.8	+16.9	12.25	33.8	100.0
20	42.2	54.0	+11.8	1.84	13.5	84.0
25	44.5	66.3	+21.8	2.84	31.7	100.0
30	52.3	48.3	-4.0	0.97	9.8	82.6
35	63.3	63.9	+0.6	3.18	19.4	97.9
40	79.7	59.3	-20.4	2.45	22.2	100.0

Table 7: **Comparison of GRPO+KL Performance on ALFWorld benchmark.** All SR values are percentages. SR@Invoke and SR@Skip are computed on validation episodes with and without skill calls, respectively.

learned policy still invokes skills in many low-yield states. This does not mean that skill calls directly cause failure, since invoked episodes may be harder or more uncertain. Rather, the result shows that sparse task reward, even with a simple skill-success bonus, is too coarse to supervise the local skill-or-skip decision. The policy can improve final task success while still failing to learn when a skill should be skipped.

Table 8 illustrates this issue with a representative ALFWorld trajectory. The model eventually succeeds, but it invokes `heat_object` for a task that only requires placing a tomato on the microwave. The redundant calls waste steps, introduce precondition errors, and make the trajectory much longer than necessary. Thus, the final success reward cannot distinguish a skill that helped from a skill that the model succeeded despite. This is the credit assignment problem at the individual skill-call level.

D Additional Selective-Invocation Baselines

Engineering baselines. We evaluate whether simple engineering changes can replace selectivity training on ALFWorld with Qwen3-8B RL-Init. *Skill-as-Context* injects all skill content directly into the prompt, *Conservative Prompt* explicitly discourages unnecessary calls, and *Explicit Skip Option* adds a universal skip skill. None improves over RL-Init w/ Skill, while SelSkill reaches substantially higher SR with fewer and more reliable invocations.

Adapted selective-invocation baselines. We adapt two prior selective-use ideas to our setting rather than treat them as direct reproductions. *Raw-Logit Gate* (Xu et al., 2025) skips skill access when the no-skill action has low mean token NLL, while *Call-or-Skip Gate* (Ross et al., 2025) trains

a same-backbone SFT invoke/skip classifier using SelSkill’s outcome-efficiency labels. Both improve execution precision but trail SelSkill in SR and SR@Invoke at similar Skill/ep.

BFCL failure analysis. Among the remaining attributable BFCL failures, 7.2% arise from incorrect skill/function decisions, 40.3% from missing API/environment knowledge, and 52.5% from downstream reasoning errors.

E Ablation on Entropy-Guided Branching

We ablate the branching-point selection strategy used for local decision-point preference construction. All conditions use the same base model, Qwen3-8B RL-Init on ALFWorld, and the same $K = 4$ free-sampling procedure. To isolate the effect of branching-point selection, we match the final amount of training data to the entropy-guided setting across all strategies.

Table 10 reports the retained local preference pairs before final size matching. Random branching retains many pairs, but only a relatively small portion of them prefer the invoke continuation. Entropy-guided branching retains a comparable number of pairs while yielding a higher invoke-preferred ratio. All-skill branching has the highest invoke-preferred ratio, but it retains substantially fewer pairs per game. This is because it attempts forks at all skill-invocation positions, while many of these positions lead the invoke and skip continuations to the same final outcome and therefore cannot form a clear outcome-efficiency preference. Thus, trying more branch points does not necessarily produce more valid preference pairs. These results suggest that entropy-guided selection can more efficiently identify positions where valid local invoke/skip comparisons can be constructed.

Table 11 further compares the downstream re-

RL-Init model: successful trajectory with redundant skill calls

Task: Put some tomato on microwave. Success in 18 steps; optimal ≈ 6 .

Correct prefix	go to fridge 1 $\rightarrow$ open fridge 1 $\rightarrow$ take tomato 3 from fridge 1. The agent correctly obtains the tomato.
Redundant invocation	Skill(heat_object) is invoked and executes, although the task only requires placing the tomato in the microwave.
State-breaking action	go to countertop 1 $\rightarrow$ move tomato 3 to countertop 1. The unnecessary skill invocation changes the subsequent behavior, and the agent puts down the tomato.
Failed invocations	The model invokes Skill(heat_object) twice more. Both calls fail because the agent is no longer holding an object, violating the skill precondition.
Recovery	go to countertop 1 $\rightarrow$ take tomato 3 from countertop 1 $\rightarrow$ Skill(heat_object) $\rightarrow$ move tomato 3 to microwave 1. The task is eventually completed, but only after several unnecessary steps.

Table 8: **Success Despite Redundant Skill Calls.** An ALFWorld trajectory where the model succeeds on a placement-only task despite unnecessary heat_object calls that introduce precondition errors and extra steps.

Method	SR ($\uparrow$)	Exec. Prec. ($\uparrow$)	Skill/ep	SR@Invoke ($\uparrow$)	SR@Skip ($\uparrow$)
No-Skill	78.9	—	0.00	—	78.9
RL-Init w/ Skill	75.8	70.9	2.55	73.7	78.8
Skill-as-Context	71.9	—	—	—	—
Conservative Prompt	71.9	—	1.41	78.0	66.7
Explicit Skip Option	70.3	—	2.34	72.4	67.3
Raw-Logit Gate	77.3	95.9	0.57	68.2	82.1
Call-or-Skip Gate	79.7	95.0	0.55	69.0	82.5
SelSkill Round1	82.8	94.1	0.66	90.7	78.8
SelSkill Round2	85.9	96.6	0.46	91.4	83.9
SelSkill Round3	86.7	100.0	0.44	97.0	83.2

Table 9: **Engineering and adapted selective-invocation baselines on ALFWorld with Qwen3-8B.** Raw-Logit Gate and Call-or-Skip Gate are adaptations of prior selective-use formulations rather than direct reproductions.

Strategy	Pairs/game($\uparrow$)	Inv.-pref. (%)
All-skill	0.18	31.0
Random	0.68	15.7
Entropy	0.57	24.1

Table 10: **Quality of retained local preference pairs before size matching.**

Condition	SR ($\uparrow$)	Skill/ep	Exec. Prec. ($\uparrow$)
RL-Init w/ Skill	75.8	2.55	70.9
All-skill	82.8	0.95	77.0
Random	80.5	0.59	96.1
Entropy-guided	82.8	0.66	94.1

Table 11: **Experimental Results on ALFWorld benchmark with Different Branching Strategies.** All values except Skill/ep are percentages.

sults after size-matched training. Entropy-guided branching achieves the same SR as all-skill branching, while the latter requires roughly three times the sampling cost. It also yields slightly higher SR than random branching. This indicates that entropy is not an exact causal criterion, but it provides useful

guidance for branching-point selection, achieving training performance close to exhaustive branching with much lower sampling cost.

F Robustness to Distractor Skills

We evaluate whether the invocation decisions of SelSkill Round2 remain robust as the skill listing expands. Starting from the standard 18-skill BFCL setting, we inject noise skills to create listings of 28, 38, and 68 skills. The noise skills are synthetically constructed to cover unrelated domains, such as calendar management, music streaming, e-commerce, and fitness tracking, with realistic when-to-use conditions that do not overlap with any BFCL evaluation task. They provide no task-relevant information and serve only as listing distractors. All other conditions remain identical to the main experiment.

Table 12 shows that SR remains stable when the skill listing expands from 18 to 38 skills, and only drops modestly when the listing is further expanded to 68 skills. This decline is likely due to additional context noise from the longer skill

Skill listing size	SR($\uparrow$)	Noise skill calls
18 skills (standard)	24.2	—
28 skills (+10 noise)	24.2	0 / 301 (0.0%)
38 skills (+20 noise)	25.4	5 / 250 (2.0%)
68 skills (+50 noise)	21.4	0 / 263 (0.0%)

Table 12: **Robustness of SelSkill Round2 under expanded skill listings.**

Setting	ALFWorld	BFCL
Backbone	Qwen3-8B/4B	Qwen3-14B
Mode	Thinking-style	Non-thinking
Initialization	RL-Init	Base
Eval decoding	Greedy	Greedy
Eval metric	Task success	Exact match
SelSkill rounds	3 (8B) / 2 (4B)	2
Learning rate	1×10^{-6}	5×10^{-6}
Max length	4096	12288
Tuning	Full-parameter fine-tuning	
β	0.1	
Optimizer	AdamW with cosine schedule	
Warmup	0.1	
Epochs	3	
Local mask	$n = 3$ post-branch assistant turns	

Table 13: **Model, evaluation, and training settings.**

listing, rather than incorrect invocations of the injected noise skills. The model almost never invokes these noise skills, indicating strong robustness to irrelevant or low-quality skills. This behavior is consistent with the skill-or-skip formulation: the model learns not only to identify potentially relevant skills, but also to skip skills that should not intervene in the current task state. These results suggest that SelSkill Round2 remains applicable under larger and noisier skill libraries.

G Experimental Details

G.1 Benchmark-Specific Setup

ALFWorld. We keep the benchmark’s thinking-style interaction format, because the agent needs to reason over environment observations before producing executable actions. After RL initialization, skills are enabled through the system prompt, which contains skill metadata and few-shot skill-call examples. The full skill body is loaded only after the model explicitly invokes the corresponding skill. Evaluation uses greedy decoding on a fixed held-out split.

BFCL. We use Qwen3-14B in non-thinking mode for BFCL. BFCL contains long multi-turn function-calling conversations, and enabling thinking substantially increases the context length during rollout collection and preference training. We

therefore use non-thinking mode to keep the interaction within the context budget. Skills are enabled by adding skill metadata and few-shot examples to the system prompt. The skill metadata follows the BFCL tool-calling format and includes the skill-use condition in the description. Evaluation uses exact scoring with greedy decoding.

G.2 Preference Pair Collection

Episode-level collection. For episode-level data, we sample $K = 10$ complete trajectories or task outputs for the same input and label them using the final benchmark outcome. When constructing episode-level pairs, we remove malformed positive samples so that the chosen side does not contain invalid action or call formats.

Local branching collection. For local data, we first run the current policy and record token-level log probabilities during generation. For each sampled instance, we form two candidate pools: the top-3 high-entropy positions following skill invocations and the top-3 high-entropy positions during ordinary generation. The former captures uncertain states after skill intervention, while the latter captures uncertain states in ordinary execution. For each selected branch point, we use interrupted rollout with $K = 4$. Specifically, we use a temporary intervention prompt only during data collection to elicit invoke and skip continuations from the same trajectory prefix at that branch point. Apart from this local collection prompt, all continuations use the same original prompt, skill listing, decoding setting, and evaluation protocol. After the local invoke/skip choice is made, generation continues normally under the current policy. During training, we restore the original prompt so that the chosen and rejected continuations are conditioned on the same original prefix.

The resulting continuations are then filtered and ordered by the outcome-efficiency rule in Section 4. As with episode-level data, we remove malformed positive samples from local pairs.

Sampling budget. The rollout hyperparameters above are chosen based on preliminary runs. We set episode-level collection to $K = 10$, and local collection to two groups of top-3 high-entropy positions with $K = 4$ rollout at each branch point, so that episode-level and local collection produce comparable numbers and proportions of valid preference pairs under a similar rollout-time budget. This avoids having one type of preference signal

Setting	GRPO	Rollout	Train	Total
ALFWorld 8B	30.0	11.7	2.7	44.4
ALFWorld 4B	27.0	7.8	1.2	36.0
BFCL 14B	—	4.6	1.8	6.4
GRPO comparison (without / with skill): 27.0 / 64.0 h				

Table 14: **Wall-clock time (hours) for the main experiments.**

dominate the training data and allows the global outcome signal and the local invoke/skip signal to be combined more evenly.

G.3 Training Rounds and Loss Masking

We iteratively collect preference data with the previous round’s model and train the next model only on that round’s pairs, without accumulating data across rounds. ALFWorld-8B runs three SelSkill rounds from **RL-Init**, while ALFWorld-4B and BFCL run two rounds.

We stop at these points as valid preference pairs become sparse. One additional round reduces SR from 86.7 to 85.2, 77.3 to 73.4, and 24.2 to 22.6, respectively, consistent with preference-data saturation and overfitting.

For local pairs, we mask environment observations, skill returns, tool outputs, and tokens outside the first $n = 3$ post-branch assistant turns. All training uses full-parameter fine-tuning with the hyperparameters in Table 13.

G.4 Compute Cost

All runs use 8 NVIDIA A100 80GB GPUs. Table 14 reports wall-clock costs; SelSkill’s additional cost is offline and requires no counterfactual branching at inference time.

H Trajectory Analysis

We illustrate three invocation patterns: context pollution from mistimed calls, failures from unmet preconditions, and successful calls made at an appropriate state.

BFCL context pollution. Table 15 shows that invoking `place_stock_order` introduces a live price that overrides the user’s specified limit price and derails an otherwise correct BFCL trajectory.

ALFWorld premature call. Table 16 shows a premature `examine_with_light` call before the agent acquires the desk lamp, causing repeated failures until the step budget is exhausted.

ALFWorld correct call. Table 17 shows a successful `heat_object` call after the egg has been acquired and the skill precondition is satisfied.

Turn	Without skill (✓)	With skill (✗)
1	place_order(price=150, amount=100) → order 12446 ✓	Skill(place_stock_order) → get_stock_info(AAPL) → \$227.16 → place_order(price=227.16, amount=100) → insufficient balance ✗
2	get_order_details(12446) → correct recent order ✓	get_order_history() → old completed order 12345 get_order_details(12345) ✗
3	cancel_order(12446) ✓	cancel_order(12345) → already completed ✗
4	trading_logout() ✓	trading_logout() ✓

Table 15: **Context pollution in BFCL.** A mistimed skill call introduces an additional live price into the context, causing the model to override the user-specified limit price and derail subsequent execution.

Without skill (✓)	With skill (✗)
go to sidetable 1 examine alarmclock 1 go to desk 1 pick up desk lamp 1 go to sidetable 1 use desk lamp 1 with alarmclock 1 → task complete ✓	go to sidetable 1 examine alarmclock 1 Skill(examine_with_light, alarmclock 1) → “Nothing happens.” ✗ Skill(examine_with_light, alarmclock 1) → “Nothing happens.” ✗ go to desk 1 Skill(examine_with_light, alarmclock 1) → “Nothing happens.” ✗ <i>repeats failing skill attempts</i> → step budget exhausted ✗

Table 16: **Premature invocation in ALFWorld.** The model invokes examine_with_light before acquiring the lamp, so the skill precondition is not satisfied.

With skill (✓)	Without skill (✗)
go to countertop 1 go to countertop 2 go to cabinet 1 go to fridge 1 open fridge 1 take egg 3 from fridge 1 Skill(heat_object) → go to microwave → put egg in microwave → heat egg → retrieve hot egg go to fridge 1 move egg 3 to fridge 1 → task complete ✓	go to countertop 1 go to countertop 2 go to fridge 1 go to stoveburner 1 go to fridge 1 go to sinkbasin 1 <i>repeats navigation among countertops, stove burners, microwave, cabinets, sink, coffeemachine, and garbage can</i> go to microwave 1 <i>visits microwave but never executes the heat sequence</i> → step budget exhausted ✗

Table 17: **Correct invocation in ALFWorld.** For a task requiring a heated egg, SelSkill invokes heat_object only after the egg has been picked up, when the skill precondition is satisfied.